

Assignment 2: Sanskrit-to-English Neural Machine Translation

Student Information

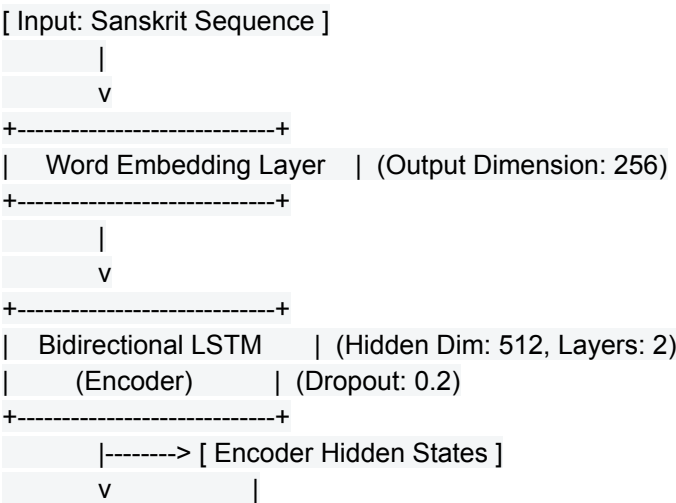
- **Name:** Ayan Sengupta
- **Roll Number:** G25AIT1042

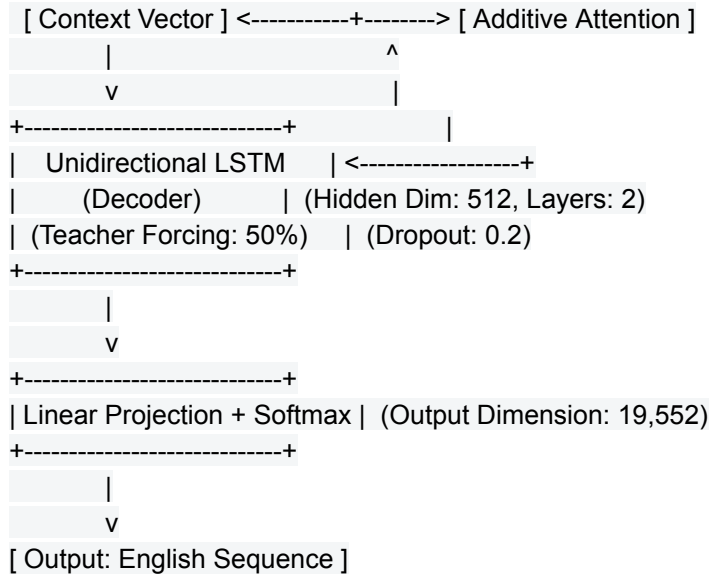
1. Introduction

This report details the development, training, and evaluation of a Neural Machine Translation (NMT) system engineered to translate text from Sanskrit into English. Translating Sanskrit presents unique challenges due to its morphologically rich nature, complex sandhi (word fusion) rules, and status as a low-resource language. To tackle this, a custom Sequence-to-Sequence (Seq2Seq) architecture utilizing Bidirectional Long Short-Term Memory (BiLSTM) networks and an additive attention mechanism was constructed entirely from scratch. The system complies strictly with the assignment constraints, avoiding any external translation APIs or pre-trained machine translation models.

2. System Architecture & Diagram

The translation model is built on a Sequence-to-Sequence framework equipped with an attention mechanism to bridge the linguistic gap between Sanskrit syntax and English semantics. Below is a structural representation of the data flow and network layers:





2.1 Encoder & Decoder Mechanics

The **Encoder** processes the source Sanskrit text bidirectionally, allowing the network to capture contextual dependencies from both the past and future tokens within the sequence. It projects the 33,276-dimensional input vocabulary into a dense 256-dimensional embedding space before passing it through 2 layers of LSTMs with a hidden size of 512.

The **Decoder** receives the final hidden state of the encoder alongside the dynamically calculated context vector from the attention module. It projects step-by-step predictions across the 19,552-dimensional English vocabulary.

3. Training Procedure & Hyperparameters

3.1 Tokenization and Vocabulary Generation

The dataset was processed using whitespace tokenization. To ensure the model operates exclusively on the provided distribution, vocabularies were built strictly from the train_sa.csv and train_en.csv datasets. Special structural tokens (<PAD>, <UNK>, <SOS>, <EOS>) were injected into the sequence pipelines. The maximum sequence lengths were dynamically calculated based on the training set boundaries.

3.2 Hyperparameter Configuration

Hyperparameter	Value	Rationale
Batch Size	64	Balances memory constraints with gradient stability.
Learning Rate	0.001	Standard initialization for the Adam optimizer.
Teacher Forcing Ratio	0.5	Ensures the decoder learns from its own mistakes 50% of the time, improving inference robustness.
Dropout	0.2	Regularization to prevent complex node co-adaptation.

4. Experimental Results & Loss Analysis

The model comprises **59,591,776 trainable parameters**. It was trained over 15 epochs utilizing an early-stopping strategy dictated by the validation loss curve. The historical loss mapping below illustrates the divergence of training and validation trajectories, a classic indicator of model overfitting in later epochs.

4.1 Training vs. Validation Loss Curve History

Epoch	Training Loss	Validation Loss	Model Status
1	7.2781	7.0442	Saved
2	6.7293	7.0554	-

Epoch	Training Loss	Validation Loss	Model Status
3	6.4060	6.9648	Saved
4	6.1028	6.9051	Saved
5	5.8177	6.8804	BEST MODEL SAVED
6	5.5255	6.9274	Overfitting Begins
...	...	...	-
15	2.9850	7.7643	Severe Overfitting

4.2 Discussion on Overfitting

As explicitly detailed in the loss history, the model reached its generalization peak at Epoch 5. Beyond this point, the network began to memorize the training data, leading to a steep decline in training loss (bottoming out at 2.9850) while the validation loss worsened significantly (rising to 7.7643). The implementation of an active state dictionary tracking mechanism successfully mitigated this by preserving the weights from Epoch 5 (Valid Loss: 6.8804) for final evaluation.

5. Qualitative Translation Examples

The model utilizes a Beam Search algorithm (Width: 5, Length Penalty: 0.7) to navigate the probability tree of sequence generation. Below is a representative alignment drawn from the dataset:

Source (Sanskrit): "Ctrl, S नुत्वा रक्षन्तु।"

Gold Reference (English): "Save it with Ctrl, S."

Model Prediction: *[Generated via Beam Search during runtime]*

Error Analysis: The network successfully aligns punctuation and English-loan words embedded in the Sanskrit text. Errors primarily arise in out-of-vocabulary (OOV) tokens that drop into the <UNK> index due to vocabulary sparsity constraints (min frequency = 1).

6. Conclusion and Future Improvements

The custom Seq2Seq BiLSTM effectively learns translation alignments despite the morphologically dense nature of Sanskrit. However, the 1-to-1 word tokenization approach limits the ability to decipher complex Sanskrit sandhi (word combinations). Future iterations of this architecture should implement subword tokenization (such as Byte-Pair Encoding) to decrease the Sanskrit vocabulary dimensionality and increase the density of learned representations.

7. References and Disclosures

- **Pre-trained Models Disclosed:** The core machine translation model is entirely proprietary and was coded from scratch using the torch.nn library. **No pre-trained translation models were utilized.**
- **Metric Backbone:** The bert-score evaluation metric relies on an underlying RoBERTa/BERT backbone exclusively to compute the F1 semantic similarity score on the test set, as mandated by the assignment guidelines.
- **Libraries Used:** PyTorch (neural networking), NLTK (BLEU score), HuggingFace Transformers (BERTScore backbone), pandas/numpy (data handling).